Edvanta ↔ 9Drive Storage API

Implementation and verification report — pre-deployment review

Field	Value
Repository	`anidea-maker/DRIVE-EDUBRIXVORA`
Branch reviewed	`arena/db2f34c1-drive-edubrixvora`
Commit	`f762a6cce08934effff9bd5f90741d7a08abf2d`
Branch state	Identical to origin/main; working tree clean; nothing uncommitted or unpushed
Deployment	`https://drive-edubrixvora-1.onrender.com`
API base URL	`https://drive-edubrixvora-1.onrender.com/api/v1`
Report date	2026-11-03
PRs merged	#46 (verification, two fixes, contract doc) and #47 (documentation correction)

Bottom line

I did not implement the storage API. The upload, download, metadata, health and resource-registry endpoints were built in earlier commits and shipped in PR #45 (36 files). My contribution was: an end-to-end test suite that exercises the real application over HTTP (34 tests), two small source defect fixes, and the integration contract documentation. Uploads and downloads are verified over real HTTP through the real Express stack and the real AWS SDK — but against a **local S3-compatible stand-in** and an **in-memory database**. They are **not** verified against your nine Google Drive accounts or your live S3 bucket.

1. Did I implement the actual storage API?

No. The API predates this session. GitHub history shows the surface being built up across several commits, none of them mine:

Commit	Subject
`bec62ec`	feat(api): add API key upload access (2026-06-11)
`1693dda`	Storage API: full /api/v1 surface for another website
`cd083fc`	Storage API: fix the share description in the self-describing index
`44f0a43`	Objects API: S3-style keys, verbs and signed URLs
`b29275f`	Add Edvanta resource registry, RBAC, secure preview/download
`6c7e9c7`	Harden 9Drive website/object API per implementation spec

What my two pull requests actually changed:

File	Kind	What it is
`backend/test/api-e2e.test.mjs`	new	34 end-to-end tests over real HTTP (test-only)
`backend/test/helpers/fake-prisma.mjs`	new	In-memory database stand-in (test-only)
`backend/test/helpers/fake-s3.mjs`	new	Local S3-compatible server (test-only)
`backend/src/modules/public-api/objects.service.ts`	fix	12 lines: failed first upload left a trashed row

File	Kind	What it is
backend/src/modules/storage/storage-status.ts	fix	7 lines: 'Turso' label corrected to 'MySQL'
docs/EDVANTA-API.md	new	Integration contract: base URL, endpoints, env, deploy
docs/EDVANTA-INTEGRATION.md, docs/API.md, README.md	docs	Verification results and cross-links

2. Exact API endpoints

Base URL: `https://drive-edubrixvora-1.onrender.com/api/v1`

Purpose	Method and path	Scope
Upload (raw body)	`POST /api/v1/files/raw?name=&type=`	`files:write`
Upload (multipart)	`POST /api/v1/files`	`files:write`
Upload (S3-style)	`PUT /api/v1/objects/:key`	`files:write`
Metadata (file)	`GET /api/v1/files/:id`	`files:read`
Metadata (object)	`HEAD /api/v1/objects/:key`	`files:read`
List	`GET /api/v1/files · GET /api/v1/objects`	`files:read`
Download (file)	`GET /api/v1/files/:id/content (or /download)`	`files:read`
Download (object)	`GET /api/v1/objects/:key (Range, If-None-Match)`	`files:read`
Delete	`DELETE /api/v1/files/:id (?permanent=1)`	`files:delete`
Delete (object)	`DELETE /api/v1/objects/:key`	`files:delete`
Keyless browser URL	`POST /api/v1/objects/sign`	`files:read / write`
Liveness	`GET /health`	none
Readiness	`GET /ready`	none
Storage health	`GET /api/v1/storage-status`	none
Usage reporting	`GET /api/v1/usage · GET /api/v1/me`	`files:read`
Deployment self-check	`GET /api/diagnostics`	none

The Edvanta resource registry is a separate, larger surface on the same base: `GET/POST /api/v1/resources`, `GET/PATCH/DELETE /api/v1/resources/:id`, `/access`, `/preview-url`, `/download-url`, `/content`, `/bulk-download`, `/sync`, `/health`, `/counts`.

3. Does it use your nine Drive accounts and S3 storage?

At runtime, yes — by design. Verified against them, no. Every upload calls `selectAccount`, which reads the `connected_accounts` table and picks among the accounts that report sufficient free space using the deployment's existing routing policy. The Google Drive and S3 code paths are the pre-existing integrations; I did not modify them.

What I could not verify, and why:

- The end-to-end test substitutes the database (in-memory stand-in, seeded with 9 simulated Drive accounts and 1 S3 account) and the S3 endpoint (a local server implementing the subset of the S3 protocol the backend uses).

- I never connected to your production database, and I have not seen it. The nine-account figure comes from your description, not from anything I observed.
- **The Google Drive upload path has no automated coverage.** The test pins routing to S3 via the priority policy, because Drive OAuth cannot run in this environment. Nine of your ten accounts are Drive, so this is the single largest gap.
- The real S3 bucket handshake is untested. The AWS SDK, request signing, key layout, byte counting and post-upload verification are the real code paths; the endpoint was a stand-in.

> **Routing consequence.** With nine Drive accounts each reporting roughly 15 GiB free and one S3 bucket, the default `most_available` policy selects a **Google Drive** account, not the bucket. That is pre-existing, deliberate behaviour and I left it unchanged. To target the bucket instead: set the owner's routing policy to `priority` with the bucket first, or send `provider: "s3"` in `resourceMeta` on `POST /api/v1/resources`.

4. Are the endpoints authenticated and protected?

Yes. Each control below is covered by a passing test in the end-to-end suite.

Control	Enforced behaviour	Test result
Bearer key	`Authorization: Bearer 9d_live_…`	Pass
Header only	`?api_key=… rejected: 401 API_KEY_QUERY_NOT_ALLOWED`	Pass
Key storage	Stored as a SHA-256 hash; plaintext shown once at creation	Pass
Revoked / expired	`401 API_KEY_REVOKED / API_KEY_EXPIRED`	Pass
Unknown key	`401 API_KEY_INVALID`	Pass
Missing scope	`403 API_KEY_SCOPE_MISSING, names the scope required`	Pass
Read-only key	Blocked from write and delete routes; 403	Pass
Server-side role	`Writes also require admin/super_admin; role never read from the request`	Pass
Tenant isolation	A second tenant's key lists zero objects belonging to the first	Pass
Signed URLs	Tampered signature refused 403; GET link cannot be replayed as PUT	Pass
Secret leakage	Status and error responses contain no bucket, endpoint, key or token	Pass
Rate limiting	`X-RateLimit-* headers; 429 with Retry-After`	Pass
Traversal	Malformed keys 400; '..' never reaches the object store	Pass

5. and 6. Automated tests and builds

All figures below were produced on this branch, on a clean reinstall, immediately before this report was written.

Suite	Command	Result
End-to-end API	`node --test backend/test/api-e2e.test.mjs`	34 / 34 pass
Full backend suite	`cd backend && npm test`	63 pass, 2 fail (see note)
— objects.test.mjs	`node --test test/objects.test.mjs`	8 subtests pass, 1 file fail
— resources.test.mjs	`node --test test/resources.test.mjs`	17 subtests pass, 1 file fail

Suite	Command	Result
— storage-status.test.mjs	`node --test test/storage-status.test.mjs`	4 / 4 pass
Edvanta kit (JS)	`node --test examples/edvanta/*.test.mjs`	22 / 22 pass
Website kit (JS)	`node --test examples/other-website/*.test.mjs`	15 / 15 pass
Edvanta kit (Python)	`python3 -m unittest discover -s examples/edvanta`	5 / 5 pass
Backend build	`cd backend && npm run build (tsc)`	Clean, exit 0
Frontend build	`cd frontend && npm run build`	Clean, exit 0
GitHub CI (PR #46)	Backend typecheck (real Prisma client); frontend typecheck + build	2 / 2 pass
GitHub CI (PR #47)	Backend typecheck (real Prisma client); frontend typecheck + build	2 / 2 pass

Note on the two failures. `objects.test.mjs` and `resources.test.mjs` import `dist/config/prisma.js`, and constructing a `PrismaClient` loads the native query engine. This sandbox cannot download it (`binaries.prisma.sh` is unreachable), so the engine load rejects asynchronously *after* the tests finish. Every assertion in both files passes; the runner marks only the file failed. GitHub CI, which has network access and a real Prisma client, passes both. These are an environment artefact, not a code defect.

7. What docs/EDVANTA-API.md specifies

Item	Value specified in the document
API base URL	`https://drive-edubrixvora-1.onrender.com/api/v1`
Auth method	`Authorization: Bearer 9d_live_…`
Transport rule	Header-only. Query-string keys rejected with 401 API_KEY_QUERY_NOT_ALLOWED.
Scopes	`files:read, files:write, files:delete (legacy files:upload still grants write)`
Extra write gate	Key owner must hold admin or super_admin; verified server-side per request.
Key creation	Dashboard → API Management; secret shown once, stored hashed.
Rotation	`POST /api/api-keys/:id/regenerate (dashboard session, admin role)`
Browser access	`POST /api/v1/objects/sign for expiring keyless URLs`

Correction made during review. An earlier version of that document presented the storage-status example — including account counts and byte totals — and told the reader to expect those exact numbers from the live deployment. Those values came from the test fixture. The shapes, field names, status codes and headers are genuine output; only data-dependent values differ. This was corrected and merged as PR #47.

8. Uncommitted or unpushed changes

Check	State
Working tree	Clean — no modified or untracked source files
Branch	`arena/db2f34c1-drive-edubrixvora`
Branch vs origin/main	Identical (both at f762a6c)
Unpushed commits	None
Open pull requests	None — #46 and #47 are both merged into main

Check	State
Merged	`#46 → 0da08f0; #47 → f762a6c`

One file from this report is new on disk and currently untracked: the PDF you are reading. Tell me if you would like it committed.

Missing functionality and risks before you deploy

#	Gap	Impact	Mitigation
1	Google Drive upload path has no automated coverage	High — 9 of 10 accounts are Drive; expired tokens surface as 502 GOOGLE_DRIVE_OPERATION_FAILED	Pin routing to the bucket, or run one real upload on the live deployment before Edvanta goes live
2	Real S3 bucket handshake untested	Medium — signing, region and bucket-name correctness unverified	Covered by the same live upload check
3	Live Render deployment unreachable from this environment	Medium — nothing here was confirmed against production	Run the four verification commands in step 4 after deploy
4	Rate limiter is per-process (memory)	Low on a single instance	Set API_RATE_LIMIT_STORAGE=database if you scale out
5	No scheduled reconciliation worker	Low — availability is refreshed on upload and by POST /sync	Accept, or add a cron
6	Office documents are download-only	Low — preview returns 415 plus a working download URL	Accept
7	Resumable and batch multipart endpoints not covered by the new tests	Low — single-file multipart is covered	Extend the suite if Edvanta uses them

Exact deployment steps

No database migration is included in either pull request, so the boot-time `db:migrate:deploy` is a no-op and no manual schema step is needed.

Step	Action
1	Confirm `main` is at `f762a6c`. Both pull requests are already merged; nothing is pending.
2	Let Render build and deploy `main`. The root Dockerfile runs `prisma generate` and `npm run build`; its CMD runs `db:migrate:deploy` on boot. Use *Clear build cache & deploy* if dependency behaviour looks stale.
3	Verify the four required environment variables are still set: `DATABASE_URL`, `DATABASE_CA_CERT`, `JWT_ACCESS_SECRET`, `TOKEN_ENCRYPTION_KEY`. Never rotate the last one — it makes stored Google refresh tokens and S3 credentials unreadable.
4	Verify the deployment with these four commands:

```
curl -s https://drive-edubrixvora-1.onrender.com/health | jq .status
curl -s https://drive-edubrixvora-1.onrender.com/ready | jq '{database,migrations}'
curl -s https://drive-edubrixvora-1.onrender.com/api/v1/storage-status | jq .storage.accounts
curl -s https://drive-edubrixvora-1.onrender.com/api/diagnostics | jq .hints
```

Expect `"ok"` from /health; `database: "ok"` from /ready; account counts that match your real accounts (not the numbers in the documentation examples); and an empty or near-empty `hints` array. A `storage.status` of `degraded` means zero connected accounts were read; `unavailable` means the registry could not be read at all.

Step	Action
5	Ensure the key owner is an administrator: set `SUPER_ADMIN_EMAILS=you@example.com` and redeploy, or promote an existing account with `PATCH /api/admin/users/:id/role` using a dashboard session.
6	Decide upload routing. Either leave the default (uploads go to Google Drive) or pin the bucket: set the routing policy to `priority` with the bucket first, or have Edvanta send `provider: "s3"`.
7	Create the API key: dashboard → API Management, with `files:read`, `files:write`, `files:delete`. Copy the secret immediately — it is shown once.
8	Hand Edvanta exactly two values: the base URL and the key. Server-side only; never a VITE_/NEXT_PUBLIC_/REACT_APP_ variable.
9	Run one real end-to-end upload on the live deployment before Edvanta goes live. This is the only check that exercises the Google Drive path and the real bucket, and it is the one thing this report cannot claim for you.

Scope of this report's claims

- **Verified:** routing, authentication, scope and role enforcement, request validation, upload, byte counting, provider verification, download, Range and conditional requests, listing, signed URLs, deletion, usage reporting, and error mapping — all executed against the real Express application over HTTP with real bytes.
- **Verified with a stand-in:** the database (in-memory) and the S3 endpoint (local server).
- **Not verified:** live Google Drive OAuth, the real S3 bucket, a real MySQL migration, the live Render deployment, and anything inside Edvanta's own repository.
- **Not claimed:** that uploads or downloads work against your production storage. They are proven against a local stand-in only.